# Guida pratica a EnergiBridge

*Misurazione energetica di programmi Java e web application su macOS e Linux con IntelliJ IDEA, Spring Boot, JSP/Servlet e JMeter. Versione rivista.*

**Obiettivo:** eseguire un workload controllato, raccogliere le metriche energetiche disponibili e confrontare versioni software senza confondere potenza, energia e tempo.

## 1. Che cosa misura EnergiBridge

EnergiBridge è uno strumento a riga di comando che esegue un comando qualsiasi e, a intervalli regolari (default 200 ms), registra su un file CSV le grandezze che la piattaforma rende disponibili: potenza, frequenza, temperatura, utilizzo di CPU/GPU e memoria. Con l'opzione `--summary` stampa anche l'energia totale al termine.

Punto fondamentale: lo strumento misura **i sensori del sistema** durante l'esecuzione del comando, non isola automaticamente il consumo del singolo processo. Tutta la guida va letta con questa avvertenza.

Il supporto dipende da sistema operativo e processore. Su macOS sono indicati CPU Intel e Apple Silicon (serie M); su Linux CPU Intel e AMD, con GPU Nvidia opzionale. I sensori realmente disponibili variano da macchina a macchina e vanno verificati nel CSV prodotto.

## 2. Installazione e verifica (macOS e Linux)

EnergiBridge è open source (licenza MIT). Codice, README e binari precompilati sono sul repository GitHub: github.com/tdurieux/EnergiBridge, con i binari nella pagina Releases.

I passi di installazione cambiano da un sistema all'altro, secondo quanto indicato nel README:

| Sistema | Processori supportati | Passi aggiuntivi |
|---|---|---|
| macOS | Intel, Apple Silicon (serie M) | compilazione; nel README nessun permesso particolare (eventuale `sudo`, vedi sotto) |
| Linux | Intel, AMD; GPU Nvidia con la libreria `nvml` | permessi di lettura sui registri MSR e capability sul binario, da rifare dopo ogni riavvio |

Hai due strade, uguali su macOS e Linux.

**Opzione A: binario precompilato.** Dalla pagina Releases del repository (https://github.com/tdurieux/EnergiBridge/releases) scarica il binario per il tuo sistema. È la via più rapida e non richiede Rust.

**Opzione B: compilazione da sorgente.** Serve Rust/Cargo. Apri il Terminale (macOS o Linux) oppure quello di IntelliJ (View → Tool Windows → Terminal):

```
cargo --version
git clone https://github.com/tdurieux/EnergiBridge.git
cd EnergiBridge
cargo build --release
./target/release/energibridge --help
```

Se `cargo` non è disponibile, installa Rust da https://rustup.rs e poi esegui `source "$HOME/.cargo/env"`. Se mancano gli strumenti di compilazione Apple, usa `xcode-select --install`.

### Note per macOS

**Permessi.** Il README per macOS indica solo la compilazione, senza passaggi di configurazione. Se però il CSV risulta con metriche di potenza assenti o a zero, prova a eseguire EnergiBridge con `sudo`: su macOS alcune letture dipendono da strumenti di sistema che possono richiedere privilegi elevati. Si tratta di una verifica empirica da fare sul proprio Mac, non di un requisito documentato.

### Note per Linux

Su Linux EnergiBridge legge i contatori di energia della CPU attraverso i registri MSR, quindi servono permessi aggiuntivi. Il README prevede due passaggi, oltre alla compilazione.

**Passo 1: compila.** Servono `git`, un compilatore C e Rust (https://rustup.rs). Su Debian o Ubuntu, ad esempio, `sudo apt install build-essential git curl`; su altre distribuzioni usa i pacchetti equivalenti. Poi:

```
git clone https://github.com/tdurieux/EnergiBridge.git
cd EnergiBridge
cargo build -r
```

**Passo 2: dai accesso in lettura ai file MSR.** I permessi si perdono a ogni riavvio, quindi vanno ripetuti:

```
sudo chgrp -R msr /dev/cpu/*/msr
sudo chmod g+r /dev/cpu/*/msr
```

Se la cartella `/dev/cpu/0/msr` non esiste, di solito va caricato il modulo del kernel con `sudo modprobe msr`. Se il gruppo `msr` non esiste o il tuo utente non ne fa parte, crealo e aggiungi l'utente seguendo la documentazione della tua distribuzione. Questi due dettagli non sono nel README: verificali sul tuo sistema.

**Passo 3: dai al binario la capability `rawio`.** Va ripetuto se sposti il binario, e vale anche per un binario scaricato dalle release:

```
sudo setcap cap_sys_rawio=ep target/release/energibridge
```

**Verifica.** Esegui una misura breve e controlla che il CSV contenga valori di potenza diversi da zero:

```
./target/release/energibridge --summary -o test.csv -- sleep 5
```

Se compare un errore di permessi o il CSV è privo di metriche, ripeti i passi 2 e 3. Per le GPU Nvidia serve anche la libreria `nvml` in fase di compilazione.

### Verifica e opzioni di EnergiBridge

**Opzioni principali** (controlla sempre `--help` per la versione installata):

| Opzione | Significato |
|---|---|
| `-o, --output <FILE>` | scrive le misure nel CSV indicato |
| `--summary` | stampa l'energia totale a fine esecuzione |
| `-i, --interval <ms>` | millisecondi tra due campionamenti (default 200) |
| `-m, --max-execution <s>` | limita la durata del comando |
| `-c, --command-output <FILE>` | salva l'output del comando misurato |
| `-g, --gpu` | raccoglie anche l'uso della GPU |

Nei comandi seguenti si assume che EnergiBridge sia stato clonato in `~/EnergiBridge`. Se usi un binario scaricato, sostituisci il percorso.

**Nota per Linux.** Nella guida il comando `open nomefile.csv` è specifico di macOS: su Linux usa `xdg-open nomefile.csv` oppure apri il file con LibreOffice Calc. Gli altri comandi (`javac`, `java`, `jmeter`, `curl`) sono uguali.

## 3. Primo esperimento: una classe Java

In IntelliJ crea un progetto Java chiamato `test` e un file `src/Main.java`. Il workload esegue ripetutamente moltiplicazioni di matrici:

```
public class Main {
    public static void main(String[] args) {
        int n = 600, repetitions = 8;
        double[][] a = new double[n][n];
        double[][] b = new double[n][n];
        double checksum = 0;
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++) {
                a[i][j] = (i + j) % 100;
                b[i][j] = (i * j) % 100;
            }
        for (int r = 0; r < repetitions; r++)
            for (int i = 0; i < n; i++)
                for (int j = 0; j < n; j++) {
                    double sum = 0;
                    for (int k = 0; k < n; k++)
                        sum += a[i][k] * b[k][j];
                    checksum += sum;
                }
        System.out.println("Checksum: " + checksum);
    }
}
```

Dal terminale, nella cartella del progetto (se `Main.java` non dichiara un `package`):

```
mkdir -p out
javac -d out src/Main.java
~/EnergiBridge/target/release/energibridge \
  --summary -o java_results.csv -- java -cp out Main
open java_results.csv
```

**Attenzione.** Il tempo riportato comprende l'avvio della JVM. Per confrontare implementazioni Java usa input equivalenti, più ripetizioni e un warm-up adeguato; evita di confrontare una singola esecuzione molto breve. Il checksum stampato serve a verificare che le versioni confrontate facciano lo stesso lavoro.

## 4. Web app Spring Boot: due modalità

### Modalità A: intero ciclo di vita del server

Compila l'applicazione e avvia il JAR attraverso EnergiBridge:

```
./mvnw package -DskipTests
~/EnergiBridge/target/release/energibridge \
  -o spring_full.csv -m 120 -- java -jar target/nome-app.jar
```

Il processo resta attivo finché il server è in esecuzione. Con `-m 120` la durata viene limitata a 120 secondi, così ottieni una finestra fissa e ripetibile senza dover interrompere a mano con Ctrl+C (di cui non è garantito come venga gestito il salvataggio del CSV). Verifica il comportamento di `-m` con `--help` sulla tua versione.

Questa modalità include startup e idle: non è adatta, da sola, a stimare l'energia di un insieme di richieste HTTP.

### Modalità B: finestra di workload

1. Avvia Spring Boot **da terminale** (`java -jar target/nome-app.jar`) e attendi che sia pronto. Avviarlo da IntelliJ aggiunge il carico dell'IDE alla misura; se possibile chiudi IntelliJ prima di misurare.
2. Esegui il warm-up.
3. Misura un intervallo definito:

```
~/EnergiBridge/target/release/energibridge \
  --summary -o spring_window.csv -- sleep 60
```

Durante i 60 secondi invia richieste da un altro terminale o con JMeter. In questa modalità EnergiBridge misura i sensori del Mac durante l'intervallo e **non** isola il processo Spring Boot. Sincronizza inizio e fine del workload con la finestra e verifica che il test finisca entro di essa.

```
for i in {1..100}; do
  curl -fsS http://localhost:8080/api/items > /dev/null
done
```

Sostituisci `/api/items` con un endpoint reale: `curl` fallisce se l'endpoint non esiste o richiede autenticazione.

**Calibrazione della finestra.** Cento richieste sequenziali con `curl` terminano in pochi secondi, molto prima dei 60 della finestra: il resto è tempo idle che diluisce il risultato. Scegli la durata di `sleep` in base al tempo reale del workload (con un margine ridotto), oppure usa JMeter con un test di durata nota.

## 5. JSP e Servlet con Apache Tomcat

Per misurare l'intero ciclo di vita, avvia Tomcat in foreground sotto EnergiBridge:

```
~/EnergiBridge/target/release/energibridge \
  -o tomcat_full.csv -m 120 -- /percorso/tomcat/bin/catalina.sh run
```

Sostituisci il percorso con la tua installazione. Per misurare una finestra di richieste senza includere lo startup, avvia prima Tomcat (da terminale, con `catalina.sh run`) e usa il metodo a tempo con `sleep` illustrato sopra. Anche in questo caso la misura non è attribuibile esclusivamente a Tomcat.

## 6. Web app e JMeter: protocollo consigliato

Crea in JMeter un Test Plan con un Thread Group e una HTTP Request. Esempio: 10 thread, ramp-up di 5 secondi, 100 iterazioni per thread, URL `localhost:8080/api/items` (1000 richieste pianificate). Le richieste pianificate non coincidono necessariamente con quelle riuscite: verifica sempre completamenti ed errori.

Procedura:

1. avvia il server;
2. verifica endpoint e dati;
3. esegui il warm-up;
4. avvia la finestra EnergiBridge;
5. lancia il test JMeter;
6. registra durata, richieste riuscite, errori, throughput e latenza;
7. ripeti nelle stesse condizioni.

```
jmeter -n -t workload.jmx -l jmeter_results.jtl
```

Il file `workload.jmx` va preparato in JMeter. Se possibile, esegui JMeter su un computer separato per ridurre l'interferenza: se gira sullo stesso Mac, il suo consumo entra nella misura. Non eseguire test di carico su servizi di terzi senza autorizzazione.

## 7. Come interpretare l'output di --summary

Esempio di output:

```
Energy consumption in joules: 1.7163553524017332
for 0.22915384 sec of execution.
```

Significa: energia misurata E = 1,716 J; durata t = 0,229 s; potenza media equivalente P = E / t ≈ 7,49 W. Non significa necessariamente che il solo codice Java abbia consumato 1,716 J: il perimetro dipende dai sensori e dalla piattaforma.

| Indicatore | Interpretazione |
|---|---|
| Energia (J) | quantità totale misurata nell'intervallo |
| Tempo (s) | durata dell'esecuzione o della finestra di misura |
| Potenza media (W) | energia divisa per durata; 1 W = 1 J/s |
| Energia per richiesta (J/req) | energia misurata divisa per le richieste riuscite |
| CPU / RAM / temperatura | metriche contestuali, se presenti nel CSV; non sono energia |

## 8. Come leggere il CSV

Apri il file con `open nomefile.csv` oppure con Excel. Le intestazioni possono variare per piattaforma e versione. Il formato documentato prevede una riga per intervallo, con colonne come:

- `Delta`: millisecondi trascorsi dal campione precedente;
- `Time`: istante del campione in millisecondi;
- `CPU_FREQUENCY_n`, `CPU_TEMP_n`, `CPU_USAGE_n`: frequenza (MHz), temperatura e utilizzo per ciascun core;
- `SYSTEM_POWER (Watts)`: potenza istantanea, se il sensore è disponibile;
- `TOTAL_MEMORY`, `USED_MEMORY`, `TOTAL_SWAP`, `USED_SWAP`: memoria in byte.

Non sommare direttamente i valori in Watt: la potenza non è energia.

**Calcolo dell'energia dai campioni.** Se il CSV contiene potenza P e intervalli Δt, l'energia si stima sommando P × Δt, con Δt in secondi (quindi `Delta` / 1000). Se invece è disponibile un contatore cumulativo di energia, si calcola la differenza tra valore finale e iniziale, tenendo conto delle unità. Su Mac è probabile il primo caso.

Esempio in Python (controlla i nomi delle colonne nel tuo CSV):

```
import csv

with open("java_results.csv") as f:
    rows = list(csv.DictReader(f))

# colonna di potenza: cerca un'intestazione che contenga "POWER"
power_col = next(c for c in rows[0] if "POWER" in c.upper())

energy_j = sum(
    float(r[power_col]) * float(r["Delta"]) / 1000
    for r in rows
    if r[power_col] != ""
)
duration_s = sum(float(r["Delta"]) for r in rows) / 1000

print(f"Energia stimata: {energy_j:.3f} J")
print(f"Durata: {duration_s:.3f} s")
print(f"Potenza media: {energy_j / duration_s:.2f} W")
```

Questa è una stima per integrazione dei campioni, con un'approssimazione legata all'intervallo di campionamento. Quando disponibile e appropriato, preferisci il riepilogo `--summary` e confronta i due valori come controllo di coerenza.

## Come interpretare i risultati: procedura di lettura

Per passare dal CSV a un numero confrontabile servono cinque passi, sempre nello stesso ordine.

1. **Verifica che il run sia valido.** Controlla che il file abbia abbastanza righe (durata ÷ intervallo di campionamento, ad esempio 60 s a 200 ms ≈ 300 righe), che la colonna di potenza esista e non sia vuota o sempre a zero, e che `USED_SWAP` non cresca. Un run con sensori a zero o con il Mac in swap va scartato, non interpretato.
2. **Calcola le tre grandezze di base.** Energia totale E (joule, sommando P × Δt), durata t (secondi, sommando `Delta` ÷ 1000) e potenza media P = E / t (watt). Controlla che E coincida, a meno di piccole differenze, con il valore stampato da `--summary`.
3. **Sottrai il baseline idle.** Misura lo stesso intervallo con il Mac a riposo ( `sleep` della stessa durata, nessun workload) e sottrai la sua energia: ottieni l'energia *incrementale*, cioè quella attribuibile al workload, con le cautele descritte nei limiti del metodo.
4. **Normalizza sul lavoro svolto.** L'energia assoluta confronta solo run che fanno la stessa cosa. Dividi per le richieste riuscite (J/richiesta) oppure per il numero di esecuzioni dell'algoritmo, così versioni con carichi leggermente diversi restano confrontabili.
5. **Ripeti e riassumi.** Un solo run non basta: riporta mediana e dispersione (deviazione standard o intervallo min–max) su almeno 5–10 ripetizioni nelle stesse condizioni.

## Cosa indica ciascuna colonna

I nomi sotto sono quelli del CSV di esempio del README e possono cambiare con piattaforma e versione: controlla sempre l'intestazione del tuo file. Solo la potenza entra nel calcolo dell'energia; le altre colonne servono a capire *perché* un run è andato in un certo modo.

| Colonna | Che cosa indica | Come usarla | Attenzione |
|---|---|---|---|
| `Delta` | ms dal campione precedente | durata di ogni intervallo: moltiplicala (in secondi) per la potenza | la prima riga vale 0; gli intervalli possono non essere perfettamente regolari |
| `Time` | istante del campione (ms) | allineare il CSV ai log di JMeter o dell'applicazione | non è una durata: serve la differenza tra due valori |
| `SYSTEM_POWER (Watts)` | potenza istantanea del sistema | integrarla nel tempo per ottenere i joule | è potenza di tutto il sistema, non del solo processo; non sommare i valori |
| `CPU_USAGE_n` | utilizzo del core n (%) | verificare che il workload carichi la CPU e capire se è mono o multi-thread | per un valore unico fai la media dei core |
| `CPU_FREQUENCY_n` | frequenza del core n (MHz) | vedere se la CPU lavora a frequenza piena o ridotta | valori a 0 nelle prime righe possono indicare campioni non ancora disponibili |
| `CPU_TEMP_n` | temperatura del core n | controllare se il Mac si scalda durante la serie di prove | temperatura alta con frequenza in calo indica un possibile throttling termico |
| `USED_MEMORY` , `TOTAL_MEMORY` | memoria usata e totale (byte) | osservare l'andamento della memoria durante il workload | una crescita continua può indicare accumulo di oggetti o una perdita |
| `USED_SWAP` | swap usato (byte) | verificare che non ci sia pressione di memoria | swap in crescita compromette tempi ed energia: ripeti il run |

## Riconoscere i pattern nel profilo di potenza

Apri il CSV in Excel e disegna `SYSTEM_POWER` in funzione del tempo: la forma della curva dice spesso più del totale. Le letture sotto sono ipotesi plausibili, da verificare con le altre colonne e con ripetizioni, non diagnosi certe.

| Che cosa vedi | Interpretazione plausibile | Che cosa fare |
|---|---|---|
| Potenza alta nei primi secondi, poi più bassa | avvio della JVM, caricamento classi, compilazione JIT | fai un warm-up e misura dopo; oppure riporta separatamente startup e regime |
| Picchi brevi e ricorrenti | garbage collection o attività di sistema in background | ripeti il run con meno app aperte; confronta con il log GC della JVM |
| Potenza alta e stabile per tutto il run | workload limitato dalla CPU (come le matrici dell'esempio) | confronta energia e tempo: qui ridurre il tempo riduce quasi sempre l'energia |
| Potenza vicina al valore idle per quasi tutta la finestra | il workload non è partito, è finito presto o la finestra è troppo lunga | controlla la sincronizzazione tra `sleep` e test JMeter; accorcia la finestra |

| | | |
|---|---|---|
| Potenza che scende col passare dei run, con temperatura alta | throttling termico: la CPU riduce la frequenza per scaldarsi meno | lascia raffreddare il Mac tra le prove; guarda `CPU_FREQUENCY` e `CPU_TEMP` |
| Grande variabilità tra run identici | rumore: processi in background, Spotlight, aggiornamenti, rete | ripeti più volte, chiudi le altre app, usa la mediana |
| Primo run molto diverso dai successivi | cache fredde e JVM non ancora ottimizzata | scarta il primo run oppure registralo come "freddo" a parte |

## Esempio svolto: dal CSV ai risultati

Esempio ipotetico con soli cinque campioni, per mostrare i calcoli. In un CSV reale le righe sono centinaia.

| Campione | `Delta` (ms) | `SYSTEM_POWER` (W) | P × Δt (J) |
|---|---|---|---|
| 1 | 0 | 8,0 | 0,0 |
| 2 | 200 | 10,0 | 2,0 |
| 3 | 200 | 12,0 | 2,4 |
| 4 | 200 | 11,0 | 2,2 |
| 5 | 200 | 9,0 | 1,8 |

1. **Energia:** 0,0 + 2,0 + 2,4 + 2,2 + 1,8 = **8,4 J**.
2. **Durata:** (0 + 200 + 200 + 200 + 200) ms = **0,8 s**.
3. **Potenza media:** 8,4 J / 0,8 s = **10,5 W**.
4. **Baseline idle:** supponiamo che, a riposo e per la stessa durata, il Mac assorba in media 6 W, cioè 6 × 0,8 = 4,8 J. L'energia incrementale del workload è 8,4 − 4,8 = **3,6 J**.
5. **Per richiesta:** se il test ha completato con successo 40 richieste, si ottengono 8,4 / 40 = **0,21 J/richiesta** in totale e 3,6 / 40 = **0,09 J/richiesta** in modo incrementale. Quale dei due riportare va dichiarato nel report.

**Più ripetizioni.** Per ottenere i file `run_1.csv`, `run_2.csv`, ... ripeti la misura con una pausa tra una prova e l'altra, così il Mac si raffredda:

```
for i in 1 2 3 4 5 6 7 8 9 10; do
  ~/EnergiBridge/target/release/energibridge \
    --summary -o run_$i.csv -- java -cp out Main
  sleep 30
done
```

Poi riassumi i risultati con questo script (come sopra, verifica i nomi delle colonne nel tuo CSV):

```
import csv, glob, statistics

def energy_j(path):
    with open(path) as f:
        rows = list(csv.DictReader(f))
    col = next(c for c in rows[0] if "POWER" in c.upper())
    return sum(float(r[col]) * float(r["Delta"]) / 1000
               for r in rows if r[col] != "")

values = [energy_j(p) for p in sorted(glob.glob("run_*.csv"))]
print("Run:", len(values))
print(f"Mediana: {statistics.median(values):.2f} J")
print(f"Media: {statistics.mean(values):.2f} J")
print(f"Dev. standard: {statistics.stdev(values):.2f} J")
print(f"Min–max: {min(values):.2f} – {max(values):.2f} J")
```

Per confrontare due versioni, esegui lo script sulle serie di entrambe (cartelle diverse) e guarda se la differenza tra le mediane supera la dispersione dei singoli run.

## Errori di interpretazione da evitare

- **Attribuire tutta l'energia all'applicazione.** La misura riguarda il sistema: include IDE, browser, JMeter e processi in background.
- **Sommare i watt** o prendere la media aritmetica di `SYSTEM_POWER` come se fosse energia. Servono sempre i pesi temporali (`Delta`).
- **Confrontare solo la potenza media.** Una versione può avere potenza più bassa ma durare di più e consumare più energia.
- **Confrontare run con carichi diversi** (richieste fallite, input diversi, finestre di durata diversa) senza normalizzare.
- **Trarre conclusioni da differenze piccole.** Se la differenza tra le mediane è dello stesso ordine della dispersione tra le ripetizioni, non si può dire che una versione consumi meno.
- **Confondere uso della CPU ed energia.** Un utilizzo alto indica attività, non il numero di joule: due run con lo stesso utilizzo possono consumare in modo diverso.
- **Mescolare condizioni diverse:** Mac con e senza alimentatore, con altre app aperte, caldo o freddo, versioni di macOS diverse.
- **Ignorare il run a freddo.** Il primo run dopo l'avvio della JVM non è rappresentativo del regime: scartalo o riportalo a parte.

## 9. Esempio di confronto tra due versioni

Esempio ipotetico:

| Metrica | Versione A | Versione B |
|---|---|---|
| Energia | 120 J | 90 J |
| Tempo | 10 s | 8 s |
| Potenza media | 12 W | 11,25 W |

Riduzione di energia: (120 − 90) / 120 × 100 = 25%. La versione B consuma il 25% di energia in meno a parità di lavoro svolto.

Il confronto è valido solo se le due versioni gestiscono le stesse richieste con lo stesso esito: se ne gestiscono di diverse o falliscono in misura diversa, non è confrontabile. Inoltre una potenza media inferiore non garantisce un'energia inferiore, perché conta anche la durata.

## 10. Protocollo minimo per un report sperimentale

Documenta:

- hardware e versione di macOS;
- versione di Java e di EnergiBridge;
- sensori disponibili (colonne effettivamente presenti nel CSV);
- workload e input;
- configurazione JMeter;
- numero di ripetizioni;
- condizioni ambientali e carichi in background (alimentazione collegata, app chiuse, sospensione disattivata).

Riporta mediana, media, dispersione e risultati grezzi; controlla errori e outlier. Per le web app confronta anche latenza e throughput. Misura anche un baseline idle (stessa durata, nessun workload): sottrarlo fornisce una stima incrementale, ma non equivale a una misura perfetta del consumo della sola applicazione.

## 11. Limiti del metodo

- Il metodo con `sleep` misura una finestra del sistema, non il singolo processo.
- Startup della JVM, JIT, garbage collection, browser, IDE e generatori di carico influenzano i valori.
- I sensori e la loro granularità dipendono dal Mac e dalla versione di EnergiBridge.
- Run brevi sono dominati dal rumore: servono più ripetizioni.

## 12. Problemi frequenti

| Errore | Controllo / soluzione |
|---|---|
| `cargo: command not found` | installa Rust e carica `~/.cargo/env` |
| `No such file or directory` | controlla `pwd`, `ls` e il percorso di EnergiBridge |
| `ClassNotFoundException` | verifica package, classpath e nome della classe |
| CSV vuoto o metriche mancanti / a zero | controlla permessi (prova `sudo`), supporto hardware e durata del test |
| HTTP 404 / connection refused | controlla endpoint, porta e stato del server |
| Test JMeter con molti errori | verifica che il server sia pronto e che l'endpoint risponda con `curl` |

**Riferimenti.** Repository: https://github.com/tdurieux/EnergiBridge. Consulta README e output di `--help` per le opzioni effettivamente disponibili nella versione installata. Articolo di riferimento: Sallou, Cruz, Durieux, *EnergiBridge: Empowering Software Sustainability through Cross-Platform Energy Measurement*, arXiv:2312.13897.